

# Analysis of Sequential and Multi-thread Computing Performance Differences in Variable-length Task Sizes

**Alexander  
Mejia Tovar**  
Tecnológico de Monterrey  
Campus Estado de México  
A01803181@tec.mx

**Cassandra Vanessa  
Padilla Barovier**  
Tecnológico de Monterrey  
Campus Estado de México  
A01753547@tec.mx

**Santiago Abraham  
Ríos Palacios**  
Tecnológico de Monterrey  
Campus Estado de México  
A01802727@tec.mx

## Abstract

This document provides an analysis on how computing performance varies depending on the task execution method applied along with the size of data to be processed. The scope of this research relies only on the analysis and comparison of sequential and multi-thread processing performance through the execution of the same task with equal size and value of data for each experiment case. The task designed for this research is to perform a syntax analysis based on a custom programming language on a bunch of text files to later create a syntax-heated HTML file per text file analyzed.

**Keywords:** Sequential, Parallel, Multi-thread, Task, Data, Performance.

## 1 Introduction

For most cases, sequential programming results the most straightforward approach to solve a problem in modern developing, nonetheless, there are some specific cases that solving a problem step by step may lead to quite expensive computation, either for large data entries or applying a brute force algorithm that cannot be more optimized, considering the use of parallel or concurrent programming is a natural alternative.

As Capel (2025) describes, a sequential program consists of a set of data instructions executed in a single linear sequence, whereas in a concurrent program, there may be many instruction execution sequences with, at least, one independent flow or execution *thread* for each of the processes running in it.

Moreover, when migrating a sequential program to a concurrent paradigm, there are several nuance differences that might trigger unexpected behavior or corrupt the functionality the program successfully performs so far if not being aware of them. One crucial is the race condition; as Balasundaram and Kennedy (1989) said, it is an inadvertently introduced scenario in which there exist two or more statements that access the same memory location and at least one of them writes into the location at the same time.

However, since in this research our scope is solely aligned to analyze the computation performance of sequential

and concurrent programming when working with large data size entries from external memory files and later writing new files in new memory locations and in different times, the experiments exerted on this work will not be affected by race conditions.

## 2 Methodology

In order to compare the performance between the two programming approaches in a replicable and verifiable way, a delimited configuration of experiments was set up in each case based on the amount of data each scenario would process and the number of threads the program would generate, if applicable.

In this sense, an experiment set was delimited as shown in Table 1, where the correlation between the data size goal and the number of threads is inverse to avoid overcharging computers when running the experiment in the program; notice that all sequential experiment configurations follow the same as described in the table, yet the number of threads shall be set as one since data will be processed in the main thread of the program.

Table 1: Experiment set configuration

| N. of experiment | Data size goal | N. of threads |
|------------------|----------------|---------------|
| 1                | 1 KB           | 100           |
| 2                | 3 KB           | 50            |
| 3                | 5 KB           | 25            |
| 4                | 10 KB          | 15            |
| 5                | 30 KB          | 7             |
| 6                | 50 KB          | 5             |
| 7                | 100 KB         | 3             |

In terms of the task, each thread will process a syntax analyzer developed in the C++ programming language. Notice that the analysis scope only covers the syntax and semantic stages, meaning that the program will solely exert two main processing phases; the segmentation of the text file input into enumerated tokens for later asserting the correct order of them to match every pattern recognized into an enumerated statement through the usage of regular expressions and context-free grammar definitions. A GitHub repository can be found in the attached link<sup>1</sup> to get access to the source code and replicate the experiments described above if desirable.

<sup>1</sup>Github Repository:  
<https://github.com/a01803181-itesm/Syntax-Analyzer>

Additionally, after all threads of an experiment configuration complete their task, the C++ program will write the summary results of the thread performance into a CSV file and finally create a new process to run a Python script to plot the data performance gathered from both the sequential and concurrent approach analysis using three core variables: time (in seconds), data processed (in bytes), and the total number of text files analyzed.

Once the results of the performance summary of all the experiments are gathered, the speed-up of every experiment will be calculated using Equation (1)

$$S_p = \frac{T_s}{T_p} \quad (1)$$

where:

$p$  is the number of processors (or cores)

$T_s$  is the time it takes to run the sequential version of the program

$T_p$  is the time it takes to run the parallel version of the program using  $p$  processors.

$S_p$  is the speed-up achieved by using  $p$  processors.

Observe that the values of  $T_s$  and  $T_p$  are obtained as the average obtained from three executions of the same experiment configuration, since time completion between analysis may vary due to other tasks the CPU might be managing dynamically in different times.

### 3 Results

The following results present a comparative visualization between the sequential and parallel implementation of the C++ syntax analyzer.

From Figure 1 to Figure 7, a contrast representation of the performance that each thread exerted during its execution can be found in terms of the data processed versus time and the number of files processed versus time. Figure 8 compares the performance differences between the sequential and parallel approach of each configuration of experiments in terms of time. Finally, Table 1 summarizes the total data and the number of files processed in each experiment configuration, while Table 2 lists the  $S_p$  values of Equation (1) obtained for each experiment configuration.

Notice that the program was run on a Dell G15 5530 computer machine with a 13th Gen Intel(R) Core(TM) CPU model i7-13650HX (2.60 GHz), a 16 GB RAM, and a 64-bit Windows 11 Operative System, therefore there might be slight differences when replicating the experiment depending on the machine computer is being used, as well as the CPU utilization percentage other OS or user tasks are consuming at the time the experiments are being processed.

#### 3.1 Graphics

This subsection lists the correlations between the number of files and the data processed per thread in each experiment configuration case.

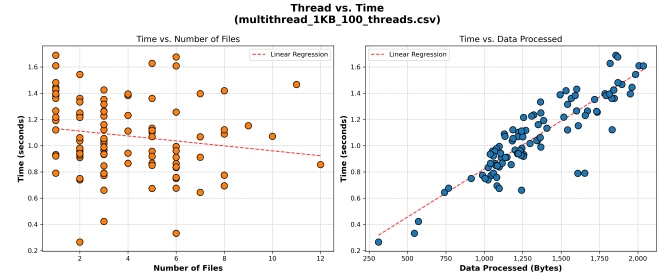


Figure 1: Experiment 1, concurrent execution with 100 threads generated, each thread processing 1 KB of data.

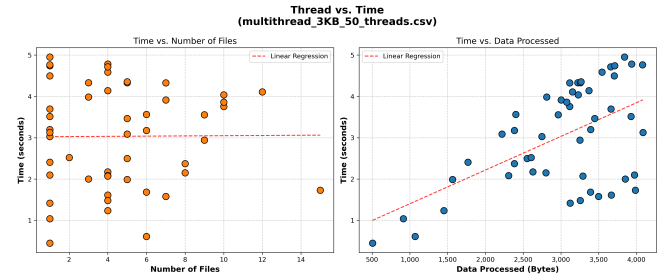


Figure 2: Experiment 2, concurrent execution with 50 threads generated, each thread processing 3 KB of data.

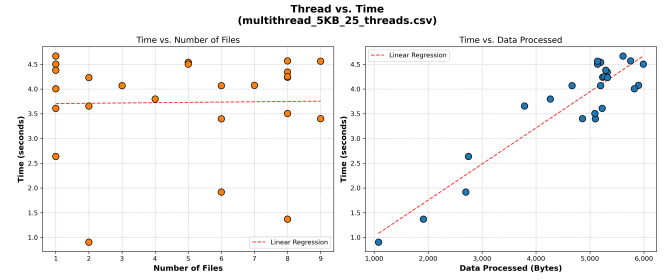


Figure 3: Experiment 3, concurrent execution with 25 threads generated, each thread processing 5 KB of data.

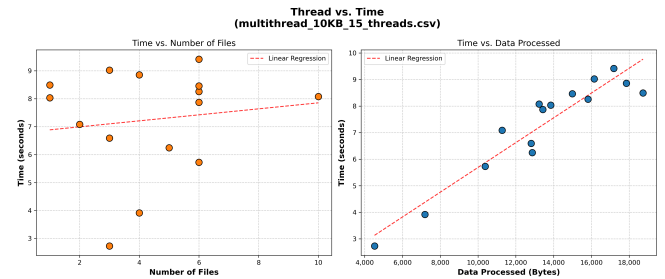


Figure 4: Experiment 4, concurrent execution with 15 threads generated, each thread processing 10 KB of data.

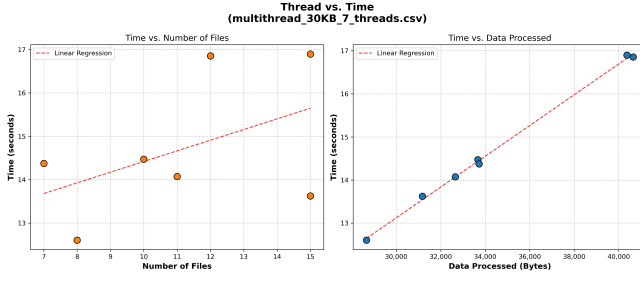


Figure 5: Experiment 5, concurrent execution with 7 threads generated, each thread processing 30 KB of data.

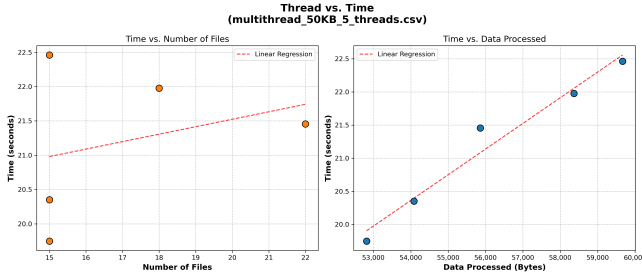


Figure 6: Experiment 6, concurrent execution with 5 threads generated, each thread processing 50 KB of data.

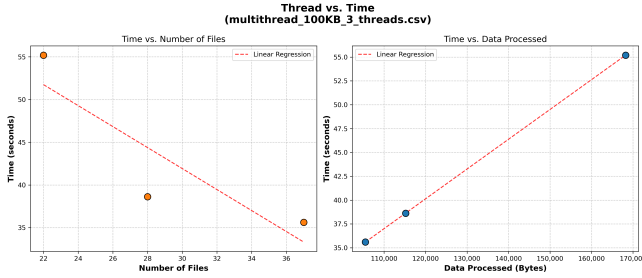


Figure 7: Experiment 7, concurrent execution with 3 threads generated, each thread processing 100 KB of data.

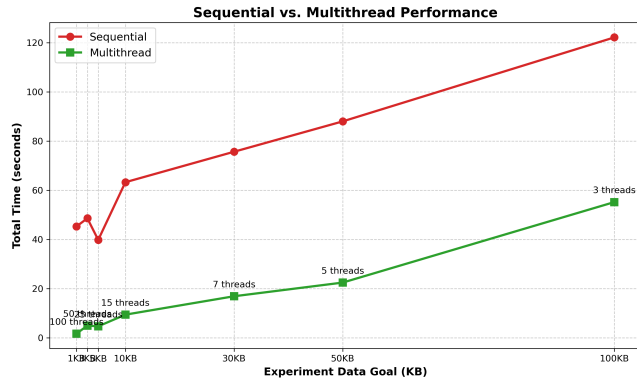


Figure 8: Concurrent and Sequential execution vs Time per experiment configuration.

### 3.2 Tables

This subsection presents the summaries of all the experiment configurations in terms of the total number of files and the total data processed, as well as the Equation (1) variable values and the  $S_p$  value obtained from each experiment.

Table 2: Experiment configuration processing summary

| N. of experiment | Total data | N. of files |
|------------------|------------|-------------|
| 1                | 135 KB     | 393         |
| 2                | 150 KB     | 232         |
| 3                | 118 KB     | 122         |
| 4                | 200 KB     | 67          |
| 5                | 241 KB     | 79          |
| 6                | 281 KB     | 86          |
| 7                | 389 KB     | 89          |

Table 3: Equation (1) calculations per experiment configuration

| N. of experiment | $S_p$ | $T_S$       | $T_P$      | $p$ |
|------------------|-------|-------------|------------|-----|
| 1                | 7.66  | 50.7 sec.   | 6.62 sec.  | 8   |
| 2                | 5.19  | 83.66 sec.  | 16.11 sec. | 8   |
| 3                | 4.3   | 54.27 sec.  | 12.63 sec. | 8   |
| 4                | 6.7   | 74.26 sec.  | 11.08 sec. | 8   |
| 5                | 4.41  | 77.5 sec.   | 17.57 sec. | 7   |
| 6                | 3.82  | 89.63 sec.  | 23.44 sec. | 5   |
| 7                | 2.17  | 126.75 sec. | 58.34 sec. | 3   |

## 4 Discussion

The experimental results listed above highlight significant insights into the performance behaviors of sequential versus concurrent execution. By analyzing the generated visualizations and performance metrics, two primary conclusions can be drawn; data size correlation and I/O bottlenecks.

### 4.1 Data Size Correlation

Initial assumptions might suggest that the total execution time of a syntax analyzer would depend on the number of files processed. However, as observed in Figures 1 to 7, all scatter plots generated from concurrent experiments demonstrate a highly scattered distribution and a weak correlation between the number of files and the thread execution time.

Conversely, when the execution time is mapped against the data processed in bytes, all plots reveal a strong, almost perfect linear correlation that tends to be quite similar to the linear regression trend drawn along all the figures.

This validates that the computational complexity of the syntax analyzer task scales linearly with the data volume  $O(bytes)$ , leaving the number of files as an irrelevant variable to the completion time of the task.

### 4.2 I/O Bottleneck

An unexpected observation from Table 3 is that the concurrent approach outperformed the sequential approach in all the

scenarios tested ( $S_P > 1$ ). In particular, Experiment 1 (100 threads processing 100 KB each) set the highest speed-up ratio of the entire experiments ( $S_P = 7.66$ ), by processing 393 files in just 6.62 seconds compared to the 50.7 seconds required sequentially.

While conventionally parallel programming degrades its performance when a massive amount of threads are created for small tasks due to its logarithmic spike behavior, this experiment acted as an exception, and the possible explanation may rely on I/O file operations.

In the sequential approach, the CPU might have remained idle while waiting for I/O operations (like opening and closing text files) for each of the 393 files, whilst in the concurrent approach, the OS utilized the concurrent context to process the data on active threads while blocking threads waited for disk I/O. Thus, the multi-thread approach effectively surpassed the hardware I/O bottleneck and, therefore, overcame the natural thread creation overhead of concurrent programming.

Additionally, the performance gained on the experiments that were able to be obtained by implementing concurrent programming can be calculated using Amdahl's law (2). According to Hennessy and Patterson (2012) book, Amdahl's law states that the performance improvement to be achieved by using some faster mode of execution is limited by the fraction of time the faster mode can be used.

$$S = \frac{1}{(1 - f) + \frac{f}{p}} \quad (2)$$

where:

$S$  is the the theoretical speedup of the whole task

$f$  is the proportion of the task that can be improved

$1 - f$  is the proportion of the task that remains sequential

$p$  is the number of cores added.

According to this statement, in Experiment 1, a maximum speed-up of  $S_P = 7.66$  was achieved using  $p = 8$  hardware cores. By substituting these values into Amdahl's Law (2),  $f$  can be solved:

$$\begin{aligned} S &= \frac{1}{1 - f + \frac{f}{p}} \implies S = \frac{1}{\frac{p}{p} - \frac{pf}{p} + \frac{f}{p}} \\ S &= \frac{p}{p - pf + f} \implies S = \frac{p}{p + f(1 - p)} \\ Sp + Sf(1 - p) &= p \implies f = \frac{p - Sp}{S(1 - p)} \\ f &= \frac{8 - 7.66(8)}{7.66(1 - 8)} \implies f \approx 0.993 \end{aligned}$$

This calculation demonstrates that approximately 99.3% of the designed C++ syntax analyzer was successfully parallelized, proving that for this kind of task the concurrent approach is more suitable.

## 5 Conclusions

In conclusion, after the tests performed, it can be stated that in this project, the sequential implementation is much slower than the parallel implementation in terms of time. Furthermore, the number of files is irrelevant; what actually affects the execution time is the size of the files. For example, if we distribute the workload of each thread by the number of files, one thread might have five very large files, while another has five almost empty files. This will cause some threads to finish earlier and therefore be idle for a while. Therefore, we can say that the number of files per thread is not relevant in the test. Consequently, it does not affect performance.

Furthermore, the number of threads used was not a factor in this case, because even with 100 threads, comparing the time it took to analyze the files with the sequential implementation, the threads were more efficient. This may be because the sequential implementation required each file to be entered individually, increasing processing time. In contrast, creating 100 different threads, each executing the program, reduced this time (although creating the threads themselves increased the overall time).

Therefore, we could predict that the same behavior might be reflected in other projects, although it is worth mentioning that the sequential implementation is linear and the parallel implementation is logarithmic. This means that if the project is smaller in scale, the sequential implementation might be more useful, although from a scalable perspective, the parallel implementation might be more advisable due to the efficiency it provides.

## References

- Vasanth Balasundaram and Ken Kennedy. 1989. Compile-time detection of race conditions in a parallel program. In *Proceedings of the 3rd International Conference on Supercomputing*, ICS '89, pages 175–185, New York, NY, USA. Association for Computing Machinery.
- M.I. Capel. 2025. *Concurrent and Real-time Programming: Principles and Applications*. Computer Science. Ibergarceta Publicaciones, , Madrid, Spain.
- John L Hennessy and David A Patterson. 2012. *Computer architecture: a quantitative approach*. Elsevier, 225 Wyman Street, Waltham, MA 02451, USA.